

Module 16: SQL JOINS (Most Important SQL Topic)

SQL JOINS are used to combine data from multiple tables based on related columns. JOINS are among the most frequently used SQL concepts in Data Analytics, Business Intelligence, Reporting, and SQL Interviews.



Introduction

In a relational database, data is usually stored across multiple tables.

Example:

Employees Table

EmployeeID	EmployeeName	DepartmentID
101	Kunal	1
102	Rahul	2
103	Aman	1

Departments Table

DepartmentID	DepartmentName
1	IT
2	HR

Question:

Which department does each employee belong to?

The answer requires combining both tables.

This is where JOINS are used.



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand why JOINS are needed

- ✓ Use INNER JOIN
 - ✓ Use LEFT JOIN
 - ✓ Use RIGHT JOIN
 - ✓ Use FULL OUTER JOIN
 - ✓ Use SELF JOIN
 - ✓ Use CROSS JOIN
 - ✓ Understand join execution
 - ✓ Solve interview questions involving JOINS
-



Table of Contents

1. What are JOINS?
 2. Why JOINS are Needed
 3. Relationship Between Tables
 4. Types of JOINS
 5. INNER JOIN
 6. LEFT JOIN
 7. RIGHT JOIN
 8. FULL OUTER JOIN
 9. SELF JOIN
 10. CROSS JOIN
 11. Join Execution Logic
 12. Common Mistakes
 13. Best Practices
 14. Interview Questions
 15. Business Analytics Examples
 16. Summary
 17. Practice Questions
-



1 What are JOINS?

A JOIN combines rows from two or more tables based on a related column.

Example

Employees

EmployeeID	EmployeeName	DepartmentID
101	Kunal	1
102	Rahul	2

Departments

DepartmentID	DepartmentName
1	IT
2	HR

Combined Result

EmployeeName	DepartmentName
Kunal	IT
Rahul	HR

Why JOINS Matter

Used in:

- Data Analytics
- Reporting
- Dashboard Development
- Data Warehousing
- Business Intelligence

2 Why JOINS are Needed

Database Normalization stores data in separate tables.

Example:

Instead of:

EmployeeID	EmployeeName	DepartmentName
101	Kunal	IT
102	Rahul	HR

We store:

Employees

EmployeeID	EmployeeName	DepartmentID
------------	--------------	--------------

Departments

DepartmentID	DepartmentName
--------------	----------------

Benefits:

- ✔ Less redundancy
- ✔ Better consistency
- ✔ Easier maintenance

To retrieve complete information:

JOIN

is required.

3

Relationship Between Tables

Employees

EmployeeID	DepartmentID
101	1
102	2

Departments

DepartmentID
1
2

Relationship:

Departments
|
|

▼

Employees

Primary Key

Departments:

DepartmentID

Foreign Key

Employees:

DepartmentID

4

Types of JOINS

Join Type	Description
INNER JOIN	Matching records only
LEFT JOIN	All left + matching right
RIGHT JOIN	All right + matching left
FULL OUTER JOIN	All records from both tables
SELF JOIN	Join table with itself
CROSS JOIN	Every combination

Sample Data

Employees

EmployeeID	EmployeeName	DepartmentID
101	Kunal	1
102	Rahul	2
103	Aman	3

EmployeeID	EmployeeName	DepartmentID
104	Priya	NULL

Departments

DepartmentID	DepartmentName
1	IT
2	HR
4	Finance

5 INNER JOIN

Returns only matching records from both tables.

Syntax

```
SELECT Columns
FROM Table1
INNER JOIN Table2
ON Table1.Column = Table2.Column;
```

Example

```
SELECT EmployeeName,
       DepartmentName
FROM Employees E
INNER JOIN Departments D
ON E.DepartmentID = D.DepartmentID;
```

Matching Logic

Employees:

```
1
2
```

```
3
NULL
```

Departments:

```
1
2
4
```

Matches:

```
1
2
```

Only.

Output

EmployeeName	DepartmentName
Kunal	IT
Rahul	HR

Visual Representation

Employees n Departments

Intersection only.

LEFT JOIN

Returns:

```
All rows from Left Table
+
Matching rows from Right Table
```

Syntax

```
SELECT Columns
FROM Table1
LEFT JOIN Table2
ON Condition;
```

Example

```
SELECT EmployeeName,
       DepartmentName
FROM Employees E
LEFT JOIN Departments D
ON E.DepartmentID = D.DepartmentID;
```

Output

EmployeeName	DepartmentName
Kunal	IT
Rahul	HR
Aman	NULL
Priya	NULL

Why?

LEFT JOIN keeps all employees.

Even if no department exists.

Visual Representation

```
Entire Employees Table
+
Matches from Departments
```

7 RIGHT JOIN

Returns:

```
All rows from Right Table
+
Matching rows from Left Table
```

Example

```
SELECT EmployeeName,
       DepartmentName
FROM Employees E
RIGHT JOIN Departments D
ON E.DepartmentID = D.DepartmentID;
```

Output

EmployeeName	DepartmentName
Kunal	IT
Rahul	HR
NULL	Finance

Why?

Finance exists in Departments but has no employees.

RIGHT JOIN still includes it.

8 FULL OUTER JOIN

Returns all records from both tables.

Example

```
SELECT EmployeeName,  
       DepartmentName  
FROM Employees E  
FULL OUTER JOIN Departments D  
ON E.DepartmentID = D.DepartmentID;
```

Output

EmployeeName	DepartmentName
Kunal	IT
Rahul	HR
Aman	NULL
Priya	NULL
NULL	Finance

Visual Representation

```
Employees  
U  
Departments
```

Complete union.



SELF JOIN

A table joins itself.

Used for hierarchical data.

Example

Employees

EmployeeID	EmployeeName	ManagerID
1	Kunal	NULL
2	Rahul	1

EmployeeID	EmployeeName	ManagerID
3	Aman	1

Query

```
SELECT E.EmployeeName,  
       M.EmployeeName AS ManagerName  
FROM Employees E  
LEFT JOIN Employees M  
ON E.ManagerID = M.EmployeeID;
```

Output

EmployeeName	ManagerName
Kunal	NULL
Rahul	Kunal
Aman	Kunal

Uses

- Employee hierarchy
- Organizational charts
- Category hierarchy

10 CROSS JOIN

Returns every possible combination.

Example

Colors

Color
Red
Blue

Sizes

Size

S

M

Query

```
SELECT *
FROM Colors
CROSS JOIN Sizes;
```

Output

Color	Size
Red	S
Red	M
Blue	S
Blue	M

Formula

```
Rows = Table1 × Table2
```

Uses

- Product combinations
- Testing
- Matrix generation

1

1

Join Execution Logic

Query:

```
SELECT *
FROM Employees E
```

```
INNER JOIN Departments D
ON E.DepartmentID = D.DepartmentID;
```

Execution:

1. Read Employees
2. Read Departments
3. Match ON condition
4. Return result

ON Clause

Most important part.

Example

```
ON E.DepartmentID = D.DepartmentID
```

Defines matching rule.

1 2 Common Mistakes

Missing ON Clause

Bad:

```
SELECT *
FROM Employees
INNER JOIN Departments;
```

Error.

Wrong Join Condition

Bad:

```
ON EmployeeID = DepartmentID
```

Incorrect results.

Ambiguous Column Names

Bad:

```
SELECT DepartmentID
FROM Employees
JOIN Departments
```

Error.

Correct:

```
SELECT E.DepartmentID
```

Using Wrong Join Type

Sometimes INNER JOIN removes required rows.

Consider LEFT JOIN instead.

1 **3** Best Practices

Always Use Aliases

Good:

```
Employees E
Departments D
```

Select Required Columns

Avoid:

```
SELECT *
```

in production.

Use Proper Join Conditions

Verify relationships carefully.

Understand Data Before Joining

Know:

- Primary Key
 - Foreign Key
-

Use LEFT JOIN for Missing Data Analysis

Very common in analytics.

Professional Query

```
SELECT E.EmployeeID,  
       E.EmployeeName,  
       D.DepartmentName  
FROM Employees E  
LEFT JOIN Departments D  
ON E.DepartmentID = D.DepartmentID;
```

1**4**

Common Interview Questions

Difference Between INNER and LEFT JOIN

INNER:

```
Matching rows only
```

LEFT:

All left rows + matches

Difference Between LEFT and RIGHT JOIN

Only table perspective changes.

What is SELF JOIN?

A table joined with itself.

What is CROSS JOIN?

Every possible combination.

Which Join is Most Used?

INNER JOIN
LEFT JOIN

1 **5** Business Analytics Examples

Employees with Departments

```
SELECT E.EmployeeName,  
       D.DepartmentName  
FROM Employees E  
INNER JOIN Departments D  
ON E.DepartmentID = D.DepartmentID;
```

Customers and Orders

```
SELECT C.CustomerName,  
       O.OrderID  
FROM Customers C  
LEFT JOIN Orders O  
ON C.CustomerID = O.CustomerID;
```

Products Without Sales

```
SELECT P.ProductName  
FROM Products P  
LEFT JOIN Sales S  
ON P.ProductID = S.ProductID  
WHERE S.ProductID IS NULL;
```

Employee Hierarchy

```
SELECT E.EmployeeName,  
       M.EmployeeName  
FROM Employees E  
LEFT JOIN Employees M  
ON E.ManagerID = M.EmployeeID;
```

KPI Dashboard Query

```
SELECT D.DepartmentName,  
       COUNT(*) AS Employees,  
       AVG(E.Salary) AS AvgSalary  
FROM Employees E  
INNER JOIN Departments D  
ON E.DepartmentID = D.DepartmentID  
GROUP BY D.DepartmentName;
```



Summary

In this module, you learned:

- ✓ INNER JOIN
 - ✓ LEFT JOIN
 - ✓ RIGHT JOIN
 - ✓ FULL OUTER JOIN
 - ✓ SELF JOIN
 - ✓ CROSS JOIN
 - ✓ Join Execution Logic
 - ✓ ON Clause
 - ✓ Business Analytics Joins
 - ✓ Interview Questions
-



Practice Questions

Theory

1. What is a JOIN?
 2. Why are JOINS needed?
 3. Difference between INNER and LEFT JOIN?
 4. Difference between LEFT and RIGHT JOIN?
 5. What is FULL OUTER JOIN?
 6. What is SELF JOIN?
 7. What is CROSS JOIN?
 8. What is the ON clause?
 9. What are aliases?
 10. Which joins are most commonly used?
-

Practical Exercises

Task 1

Create:

```
Employees
Departments
```

Perform INNER JOIN.

Task 2

Perform LEFT JOIN.

Task 3

Perform RIGHT JOIN.

Task 4

Perform FULL OUTER JOIN.

Task 5

Create Employee-Manager hierarchy using SELF JOIN.

Task 6

Create a CROSS JOIN example.

Challenge Project

Create:

```
Customers
Orders
Products
OrderDetails
```

Generate reports:

- Customer Orders
- Product Sales
- Customers Without Orders
- Products Without Sales
- Revenue by Product

using multiple JOIN types.



Next Module

→ Module 17: Subqueries

Topics Covered:

- What is a Subquery?
- Single-Row Subqueries
- Multiple-Row Subqueries
- Correlated Subqueries
- EXISTS and NOT EXISTS
- IN vs EXISTS
- Subqueries in SELECT, FROM, and WHERE
- Interview Questions